

Sistema Integral de Gestión de Eventos

Arquitectura de Software

Atributos de Calidad y Propuesta Arquitectónica

Portal Web Cliente

Portal Web Administrativo / Operativo

Aplicaciones Móviles: Cliente y Personal

Materia: Arquitectura de Software

Estudiante: Samuel Contreras

Fecha: Agosto de 2026

Índice

1	Introducción	3
2	Contexto y alcance	3
3	Interfaces del sistema	4
3.1	Portal Web del Cliente	4
3.2	Portal Web Administrativo y Operativo	4
3.3	Aplicación Móvil del Cliente	4
3.4	Aplicación Móvil del Personal	5
4	Atributos de calidad priorizados	5
5	Arquitectura propuesta	6
5.1	API Gateway y balanceador	7
5.2	Microservicios	7
6	Infraestructura de soporte	7
6.1	Redis	7
6.2	RabbitMQ / Kafka	7
6.3	Bases de datos	8
7	Servicios externos	8
7.1	Pasarela de pagos	8
7.2	Servicio de notificaciones	8
8	Cómo la arquitectura satisface los atributos de calidad	8
8.1	Consistencia – Alta	8
8.2	Disponibilidad – Alta	8
8.3	Seguridad – Alta	11
8.4	Rendimiento – Alta	11
8.5	Tiempo real – Media-Alta	11
8.6	Escalabilidad – Media	11
8.7	Trazabilidad – Media	12
8.8	Usabilidad – Baja-Media	12
8.9	Mantenibilidad – Baja-Media	12
8.10	Portabilidad – Baja-Media	12
9	Caso complejo: Gestión de Evacuación ante Emergencias	12
9.1	Objetivo	12

9.2	Actores	12
9.3	Flujo principal	13
9.4	Flujos alternativos	13
9.5	Excepciones	14
9.6	Atributos de calidad del caso	14
10	Comunicación durante una emergencia	14
11	Relación entre interfaces y microservicios	14
12	Resumen de decisiones arquitectónicas	15
13	Conclusión	15

1. Introducción

Este documento presenta una segunda versión del análisis de atributos de calidad y de la propuesta de arquitectura para el **Sistema Integral de Gestión de Eventos**. La propuesta toma como base la primera versión del proyecto, donde se planteó una organización mediana-grande que gestiona eventos masivos, como conciertos, festivales y partidos deportivos, con miles de usuarios concurrentes, especialmente durante la apertura de venta y el ingreso al recinto.

El sistema centraliza procesos de boletería, mercado secundario de entradas, administración de personal, parqueaderos y operación de eventos. En esta segunda versión se define con mayor precisión la distribución de funcionalidades entre las interfaces.

Se contemplan cuatro clientes principales:

- **Portal Web del Cliente:** compra, consulta y gestión de entradas, reventas, promociones y parqueaderos.
- **Portal Web Administrativo y Operativo:** utilizado por administradores, organizadores y trabajadores autorizados.
- **Aplicación Móvil del Cliente:** principalmente para visualizar entradas y códigos QR, información y notificaciones.
- **Aplicación Móvil del Personal:** para turnos, asistencia, instrucciones, reportes y alertas.

La primera propuesta priorizó consistencia, disponibilidad, seguridad y rendimiento. Esta versión mantiene esos atributos y aumenta la relevancia de tiempo real, trazabilidad, escalabilidad, usabilidad y portabilidad.

2. Contexto y alcance

El sistema está orientado a clientes, organizadores, administradores, supervisores, personal operativo y proveedores. Los principales dominios son:

1. Venta y gestión de boletería.
2. Mercado secundario de entradas.
3. Gestión de parqueaderos.
4. Logística y operación del evento.
5. Administración general.

6. Gestión del personal.
7. Gestión de emergencias y evacuaciones.

La arquitectura debe permitir que estos dominios evolucionen de forma relativamente independiente y que los componentes con alta demanda puedan escalar sin afectar innecesariamente a los demás.

3. Interfaces del sistema

3.1. Portal Web del Cliente

El portal web permite registro e inicio de sesión, consulta de eventos, compra de entradas, consulta de entradas adquiridas, publicación y compra de entradas revendidas, cancelaciones y devoluciones, promociones, reserva de parqueadero y recepción de notificaciones.

La lógica de negocio crítica permanece en el backend y el portal consume los servicios mediante el API Gateway.

3.2. Portal Web Administrativo y Operativo

Está dirigido a administradores, organizadores y trabajadores autorizados. Incluye:

- CRUD de eventos, usuarios, clientes, empleados y roles.
- CRUD de localidades y tipos de entrada.
- CRUD de parqueaderos y zonas.
- CRUD de proveedores, recursos logísticos y promociones.
- Planificación de eventos.
- Control de aforo.
- Gestión de incidentes.
- Gestión de emergencias y evacuaciones.
- Monitoreo del evento.

3.3. Aplicación Móvil del Cliente

Permite iniciar sesión, consultar eventos y entradas, visualizar el código QR de cada entrada, consultar información del evento y recibir notificaciones y alertas. Su función crítica es facilitar el acceso rápido a los QR durante el ingreso.

3.4. Aplicación Móvil del Personal

Permite consultar eventos asignados, turnos, horarios y áreas de trabajo; confirmar asignaciones; solicitar cambios de turno; registrar ingreso y salida; recibir instrucciones; reportar incidentes y recibir alertas de emergencia.

4. Atributos de calidad priorizados

Prioridad	Atributo	Justificación
Alta	Consistencia	Una entrada no puede pertenecer a dos propietarios ni venderse dos veces simultáneamente. También se requiere consistencia en pagos, transferencias, asignaciones y asistencia.
Alta	Disponibilidad	El sistema debe estar disponible durante ventas, ingreso masivo, operación del evento y emergencias.
Alta	Seguridad	Deben protegerse datos personales, credenciales, pagos, QR, información de empleados y permisos.
Alta	Rendimiento	Miles de usuarios pueden realizar operaciones concurrentes y las validaciones deben responder rápidamente.
Media-Alta	Tiempo real	Aforo, entradas, estado del personal, parqueaderos y emergencias deben actualizarse casi inmediatamente.
Media	Escalabilidad	Debe ser posible aumentar las instancias de los servicios con mayor demanda.
Media	Trazabilidad	Deben auditarse transferencias, pagos, accesos, turnos, operaciones administrativas y emergencias.
Baja-Media	Usabilidad	Las interfaces deben adaptarse a clientes, empleados, supervisores, organizadores y administradores.
Baja-Media	Mantenibilidad	Los cambios en un dominio no deberían afectar innecesariamente a los demás.
Baja-Media	Portabilidad	Las funcionalidades deben poder utilizarse mediante web y aplicaciones móviles.

5. Arquitectura propuesta

Se propone una arquitectura distribuida basada en **microservicios**, con API Gateway, balanceador de carga, Redis, RabbitMQ o Kafka y bases de datos independientes por dominio.

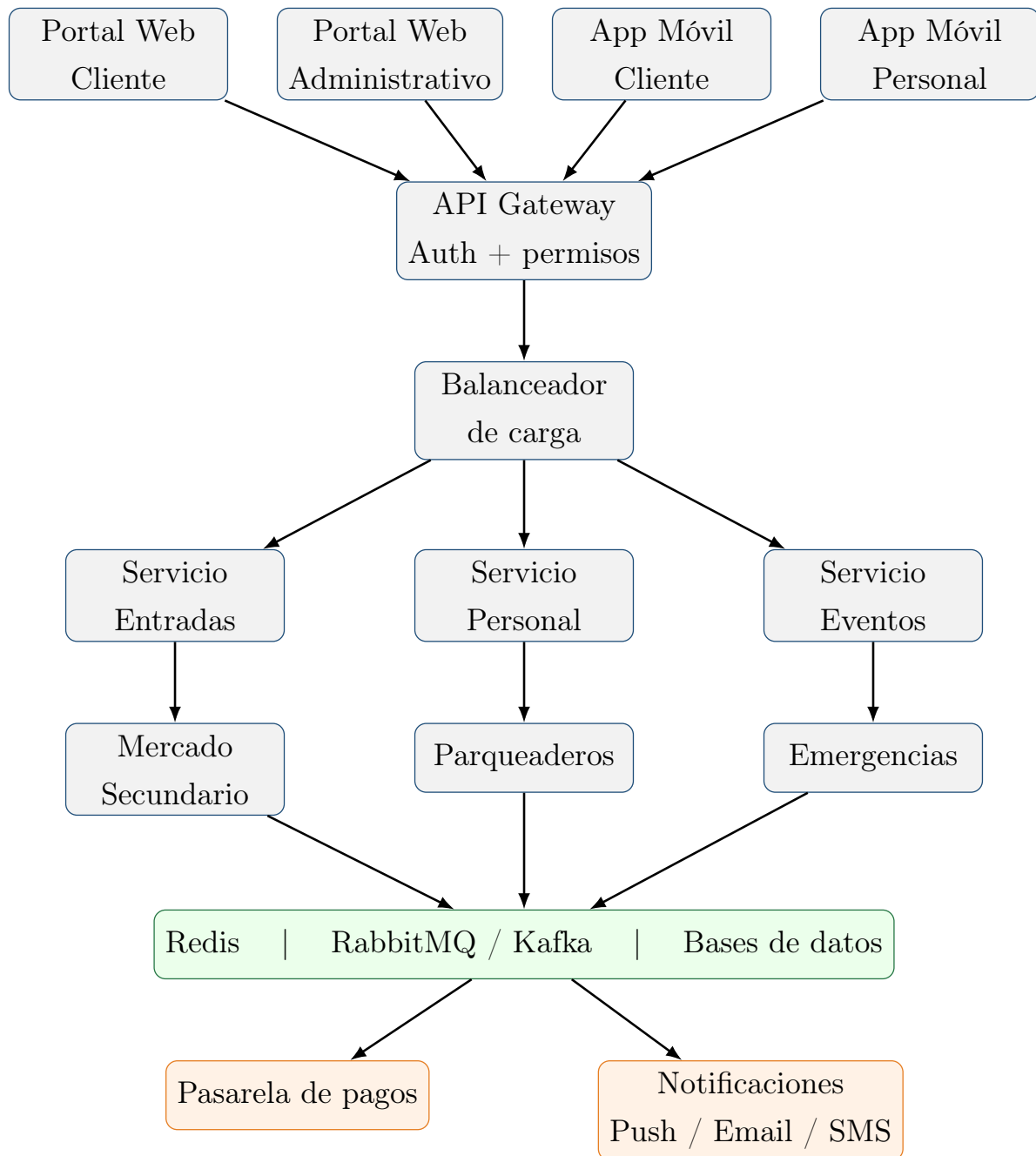


Figura 1: Arquitectura general propuesta.

5.1. API Gateway y balanceador

El API Gateway es el punto de entrada al backend. Centraliza autenticación, autorización, validación de tokens y enrutamiento. El balanceador distribuye solicitudes entre múltiples instancias y permite mantener el servicio disponible cuando una instancia falla.

5.2. Microservicios

- **Entradas:** compra, disponibilidad, validación y gestión de entradas.
- **Mercado secundario:** publicación, reventa, compra y transferencia de propiedad.
- **Personal:** empleados, roles, turnos, asignaciones y asistencia.
- **Eventos:** eventos, localidades, configuración y aforo.
- **Parqueaderos:** reservas, espacios, ingreso, salida, cobro y ocupación.
- **Emergencias:** incidentes, evacuaciones, zonas, rutas, alertas y seguimiento.

Cada microservicio puede disponer de su propia base de datos para reducir el acoplamiento.

6. Infraestructura de soporte

6.1. Redis

Redis se utiliza para información de acceso rápido y control de concurrencia. En el mercado secundario puede bloquear temporalmente una entrada mientras se procesa una compra, evitando que dos compradores completen simultáneamente la misma operación.

También puede apoyar información temporal de aforo, disponibilidad, sesiones y estados que requieran consultas rápidas.

6.2. RabbitMQ / Kafka

Las colas desacoplan operaciones asíncronas como generación de QR, notificaciones, auditoría, liquidación de pagos y propagación de alertas.

Un ejemplo es:

$$\text{ENTRADA_TRANSFERIDA} \rightarrow \text{COLA} \rightarrow \begin{cases} \text{Generar QR} \\ \text{Notificar} \\ \text{Auditar} \end{cases}$$

6.3. Bases de datos

Se plantea una base de datos independiente por dominio: entradas, personal, eventos, parqueaderos, emergencias y mercado secundario. Esto reduce el acoplamiento y permite que los servicios evolucionen de forma independiente.

7. Servicios externos

7.1. Pasarela de pagos

Se utiliza para compras, reventas, reembolsos y cobros del parqueadero. La integración debe manejar respuestas exitosas, rechazos, tiempos de espera y fallos.

7.2. Servicio de notificaciones

Envía notificaciones push, correo y SMS cuando corresponda. Ejemplos: confirmación de compra, reventa, nuevo QR, cambios de evento, turnos, asignaciones y emergencias.

8. Cómo la arquitectura satisface los atributos de calidad

8.1. Consistencia – Alta

Redis puede utilizarse para bloqueo temporal de entradas durante el checkout. La persistencia de la operación se realiza en el servicio correspondiente, evitando que una entrada tenga simultáneamente dos operaciones de compra válidas.

8.2. Disponibilidad – Alta

El balanceador y las múltiples instancias permiten redirigir solicitudes hacia instancias sanas. Esto es especialmente importante durante ventas, ingreso masivo, validación QR y emergencias.

Siguiendo el catálogo de tácticas (detección, recuperación y prevención de defectos) y patrones (redundancia, *circuit breaker*) de disponibilidad de Bass/Kazman visto en clase, la Tabla 2 identifica las tácticas y patrones aplicables a los escenarios de disponibilidad más críticos del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Falla de una instancia de validación de QR durante el ingreso masivo	Una instancia deja de responder mientras miles de asistentes ingresan; el balanceador debe redirigir el tráfico a instancias sanas sin interrupción perceptible.	Heartbeat / monitoreo (detección); redundant spare + reconfiguración (recuperación); reinicio escalado para reintegrar la instancia (reintroducción).	Redundancia activa (<i>hot spare</i>) tras balanceador de carga.
Picos de tráfico en la apertura de venta	Miles de usuarios consultan y compran de forma simultánea; el servicio de Entradas debe absorber la carga sin caerse, degradando funciones no críticas si es necesario.	Monitoreo de carga (detección); autoescalado + reconfiguración (recuperación); sala de espera virtual diseñada como estado competente, no como error (prevención); <i>graceful degradation</i> .	Redundancia activa con escalado horizontal (<i>elastic</i>) del microservicio de Entradas.
Caída del API Gateway (punto único de entrada)	La instancia activa del Gateway falla; el tráfico debe redirigirse de inmediato a otra instancia sin exponer el fallo a las cuatro interfaces.	Heartbeat / <i>health-check</i> (detección); redundant spare + reconfiguración (recuperación).	Redundancia activa (<i>hot spare</i>) + <i>Circuit Breaker</i> hacia los microservicios lentos, para evitar fallos en cascada.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Indisponibilidad de la pasarela de pagos externa	La pasarela no responde o responde con error durante una compra o reventa; la operación no debe fallar por completo.	Detección de excepciones / <i>timeout</i> ; <i>retry</i> con máximo de reintentos (recuperación); <i>graceful degradation</i> (marcar la transacción como pendiente en vez de fallar toda la operación).	<i>Circuit Breaker</i> (evita seguir enviando solicitudes a una pasarela caída).
Falla del proveedor de notificaciones durante una emergencia	El proveedor de notificaciones no responde mientras se coordina una evacuación; el protocolo de evacuación no debe detenerse por esta falla.	Detección de excepciones / <i>timeout</i> ; <i>retry</i> limitado + reconfiguración hacia un canal secundario, SMS si falla el push (recuperación); <i>graceful degradation</i> (el personal en sitio releva la comunicación).	<i>Circuit Breaker</i> + redundancia pasiva (canal secundario en espera).
Falla del nodo de Redis usado para el bloqueo de concurrencia en reventa	El nodo Redis activo se cae mientras se procesa una reventa; el sistema debe conmutar a una réplica sin permitir que dos compradores completen la misma compra.	Heartbeat / <i>health-check</i> del clúster (detección); <i>redundant spare</i> (recuperación); resincronización de estado tras el <i>failover</i> (reintroducción).	Redundancia activa/pasiva vía réplica de Redis (Redis Sentinel/Cluster).

Tabla 2: Tácticas y patrones de disponibilidad aplicados a los escenarios críticos del sistema.

En resumen, la **detección** de fallas se apoya en heartbeat/health-check, monitoreo de

carga y detección de excepciones/timeout; la **recuperación** combina redundant spare, reconfiguración, reintentos (retry), degradación controlada (graceful degradation), reinicio escalado y resincronización de estado; y la **prevención** se refleja en diseñar de entrada estados competentes para los picos de tráfico esperados (sala de espera virtual) en vez de tratarlos como una excepción. A nivel de patrones, la **redundancia activa (hot spare)** detrás de un balanceador de carga protege el Gateway, el servicio de Entradas y el clúster Redis; el **Circuit Breaker** protege las integraciones más propensas a fallar por causas externas (pasarela de pagos, proveedor de notificaciones, y las llamadas del Gateway hacia microservicios lentos), evitando que una falla externa se propague en cascada al resto del sistema.

8.3. Seguridad – Alta

El API Gateway centraliza autenticación y autorización. Se propone control de acceso basado en roles:

Rol	Ejemplos de permisos
Cliente	Consultar eventos, comprar y gestionar sus entradas.
Empleado	Consultar turnos, registrar asistencia y reportar incidentes.
Supervisor	Supervisar personal, aprobar cambios y operar protocolos.
Organizador	Gestionar eventos, personal y operación.
Administrador	Administrar usuarios, roles y configuración.

8.4. Rendimiento – Alta

El balanceador, las instancias independientes, Redis y las colas permiten distribuir la carga y ejecutar operaciones pesadas de manera asíncrona.

8.5. Tiempo real – Media-Alta

Aforo, entradas, parqueaderos, personal y emergencias requieren actualizaciones cercanas al tiempo real. Redis, colas y mecanismos de comunicación en tiempo real, como Web-Sockets, pueden utilizarse para estos escenarios.

8.6. Escalabilidad – Media

Los servicios se pueden escalar individualmente. Durante una preventa puede crecer el servicio de Entradas, mientras que durante el ingreso puede aumentar principalmente la

capacidad de validación.

8.7. Trazabilidad – Media

Se deben registrar operaciones como transferencias, compras, reembolsos, cambios de turno, accesos, modificaciones administrativas y emergencias.

8.8. Usabilidad – Baja-Media

Cada tipo de usuario posee una interfaz especializada: cliente, personal y administración. Esto evita presentar a cada usuario funciones que no necesita.

8.9. Mantenibilidad – Baja-Media

La separación en microservicios permite modificar un dominio sin modificar directamente los demás. Esto facilita pruebas, despliegues y evolución independiente.

8.10. Portabilidad – Baja-Media

Los servicios del backend son consumidos mediante APIs, por lo que pueden ser utilizados desde el portal web del cliente, portal administrativo y aplicaciones móviles sin duplicar la lógica de negocio.

9. Caso complejo: Gestión de Evacuación ante Emergencias

Este caso de uso se propone como un proceso complejo porque integra varios dominios y requiere disponibilidad, seguridad, rendimiento, tiempo real y trazabilidad.

9.1. Objetivo

Permitir que un usuario autorizado active, coordine, supervise y finalice un protocolo de evacuación utilizando información de zonas, aforo, salidas, rutas y personal operativo.

9.2. Actores

- Supervisor de emergencia.
- Organizador.
- Personal de seguridad.
- Personal médico.

- Asistentes.
- Servicio de notificaciones.

9.3. Flujo principal

1. El supervisor inicia sesión en el módulo de emergencias.
2. Selecciona “Activar evacuación”.
3. El sistema solicita el tipo y ubicación de la emergencia.
4. El supervisor registra la información.
5. El sistema identifica las zonas potencialmente afectadas.
6. Consulta el aforo actual.
7. Identifica las salidas disponibles.
8. Determina las rutas configuradas.
9. Presenta las rutas y zonas recomendadas.
10. El supervisor confirma el protocolo.
11. El sistema notifica al personal operativo.
12. El sistema envía instrucciones a los asistentes.
13. El sistema monitorea las zonas.
14. El supervisor consulta el avance y atiende alertas.
15. El supervisor confirma la finalización y el sistema registra la emergencia.

9.4. Flujos alternativos

Ruta bloqueada: si una salida deja de estar disponible, el sistema la retira de las rutas recomendadas, busca una alternativa y actualiza las instrucciones.

Zona congestionada: el sistema genera una alerta y el supervisor puede seleccionar una ruta alternativa.

Evacuación parcial: solamente se notifican y controlan las zonas afectadas.

Cambio del nivel de emergencia: el sistema vuelve a evaluar zonas y rutas cuando aumenta la gravedad.

9.5. Excepciones

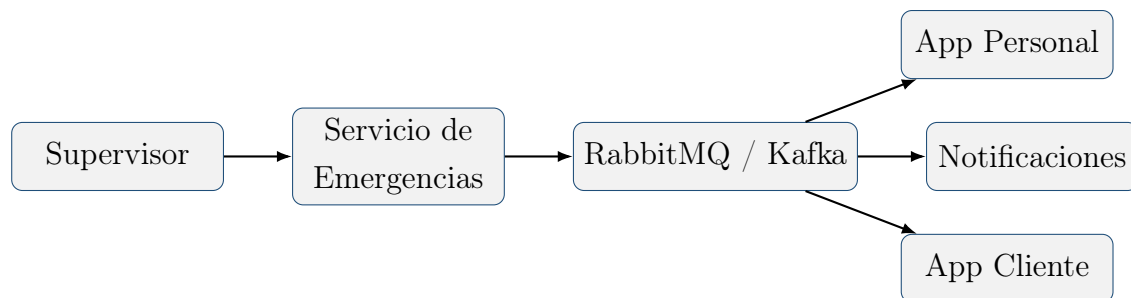
- No existen rutas disponibles.
- Se pierde comunicación con el monitoreo.
- Falla el servicio de notificaciones.
- El usuario no tiene permisos.
- Se pierde temporalmente la conexión de una aplicación.

9.6. Atributos de calidad del caso

- **Disponibilidad:** el servicio debe permanecer operativo durante la emergencia.
- **Rendimiento:** las alertas deben procesarse rápidamente.
- **Tiempo real:** el estado de las zonas debe actualizarse con baja latencia.
- **Seguridad:** solo usuarios autorizados pueden activar o modificar el protocolo.
- **Trazabilidad:** las acciones deben quedar registradas.

10. Comunicación durante una emergencia

Un escenario representativo es un concierto en el que se detecta un incendio en una zona. El supervisor activa el protocolo desde el portal administrativo. El servicio de emergencias consulta aforo, zonas y rutas y publica eventos para notificar al personal y a los asistentes.



11. Relación entre interfaces y microservicios

Servicio	Interfaces	Responsabilidad
Entradas	Web Cliente, App Cliente	Compra, disponibilidad, validación y gestión.

Mercado Secundario	Web Cliente	Publicación, compra y transferencia.
Personal	Web Administrativo, App Personal	Empleados, turnos, asignaciones y asistencia.
Eventos	Web Administrativo, Web Cliente, Apps	Gestión y consulta de eventos.
Parqueaderos	Web Cliente, Web Administrativo, Apps	Reservas, espacios, cobros y ocupación.
Emergencias	Web Administrativo, App Personal, App Cliente	Activación, coordinación, alertas y seguimiento.

12. Resumen de decisiones arquitectónicas

Decisión	Problema	Atributos
Microservicios	Separación de dominios y escalamiento	Escalabilidad, mantenibilidad, rendimiento
API Gateway	Punto central de acceso y seguridad	Seguridad, mantenibilidad
Balanceador	Distribución de solicitudes	Disponibilidad, rendimiento
Redis	Acceso rápido y bloqueo temporal	Consistencia, tiempo real, rendimiento
RabbitMQ/Kafka	Procesamiento asíncrono	Rendimiento, escalabilidad
BD por servicio	Evitar acoplamiento de datos	Mantenibilidad, escalabilidad
Notificaciones	Comunicación desacoplada	Rendimiento, tiempo real
Pasarela de pagos	Transacciones externas	Seguridad, consistencia
Interfaces especializadas	Adaptación al usuario	Usabilidad, portabilidad

Tabla 4: Resumen de decisiones arquitectónicas.

13. Conclusión

La arquitectura propuesta conserva las decisiones fundamentales de la primera versión, pero las adapta a una definición más concreta del sistema: un portal web para clientes, un portal web administrativo y operativo, una aplicación móvil para clientes y una aplicación móvil para el personal.

La separación de interfaces permite que cada tipo de usuario tenga las funcionalidades que

necesita, mientras que el backend mantiene la lógica de negocio centralizada en servicios independientes.

Los atributos de calidad prioritarios siguen siendo consistencia, disponibilidad, seguridad y rendimiento. Estos se soportan mediante microservicios, API Gateway, balanceador de carga, Redis, colas de mensajes, bases de datos independientes y servicios externos de pago y notificaciones.

La incorporación de aplicaciones móviles aumenta la importancia de tiempo real, usabilidad y portabilidad. Asimismo, la gestión de evacuación ante emergencias constituye un caso complejo adecuado para demostrar cómo las decisiones arquitectónicas responden a atributos de calidad concretos.